

GreenBite — Informe de Pruebas Unitarias

Desarrollo Fullstack III · Evaluación Parcial N°3

1. Introducción

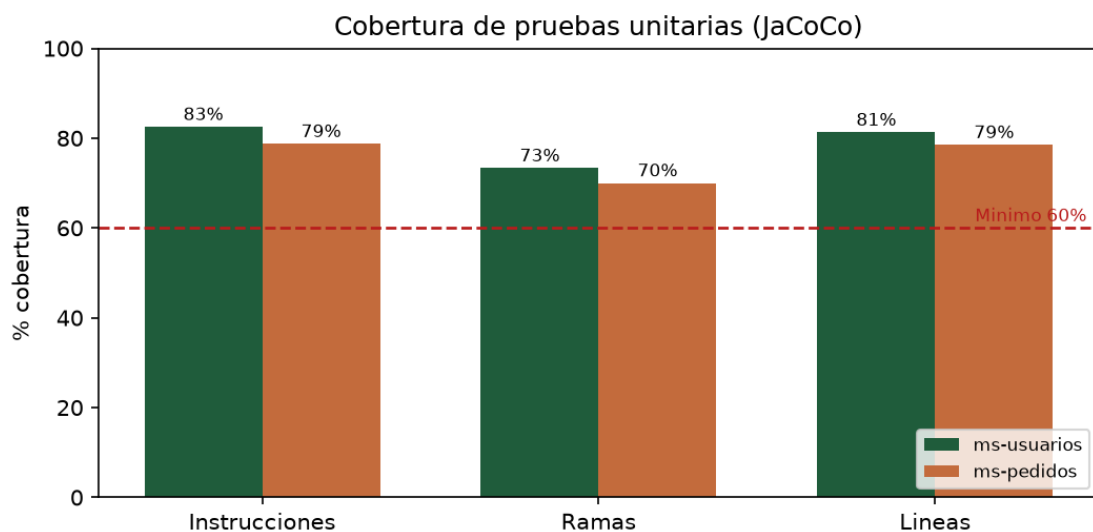
Los microservicios **ms-usuarios** y **ms-pedidos** (Java 17 + Spring Boot) se prueban con **JUnit 5** y **Mockito**. Las pruebas son unitarias: se mockean los repositorios (Spring Data JPA) y demás dependencias para validar la lógica de negocio de forma aislada, sin base de datos. La cobertura se mide con **JaCoCo**, que genera reportes HTML y CSV en `target/site/jacoco/`.

2. Resumen de resultados

Microservicio	N° pruebas	Resultado	Instr.	Ramas	Líneas
ms-usuarios	24	100% OK	82.5%	73.3%	81.3%
ms-pedidos	21	100% OK	78.7%	70.0%	78.5%
TOTAL	45	100% OK	—	—	—

Se ejecutaron **45 pruebas unitarias** en total (24 en ms-usuarios y 21 en ms-pedidos), todas exitosas. Ambos microservicios **superan el mínimo del 60%** de cobertura exigido por la rúbrica.

3. Gráfico de cobertura



4. Cobertura por clase

ms-usuarios

Clase	Paquete	Instr.	Cobertura
UsuarioService	usuarios.service	207	99%
Usuario	usuarios.entity	66	70%
UsuarioController	usuarios.web	45	100%
GlobalExceptionHandler	usuarios.exception	43	100%
JwtService	usuarios.security	43	100%
AppConfig	usuarios.config	42	0%
ApiException	usuarios.exception	34	100%

OpenApiConfig	usuarios.config	31	0%
UserResponse	usuarios.dto	27	100%
UpdateRequest	usuarios.dto	12	100%
RegisterRequest	usuarios.dto	12	100%
AuthResponse	usuarios.dto	9	100%
LoginRequest	usuarios.dto	9	100%
MsUsuariosApplication	usuarios	8	0%
HealthController	usuarios.web	7	100%

ms-pedidos

Clase	Paquete	Instr.	Cobertura
PedidoService	pedidos.service	154	100%
Pedido	pedidos.entity	76	68%
GlobalExceptionHandler	pedidos.exception	43	100%
PedidoController	pedidos.web	41	100%
AppConfig	pedidos.config	38	0%
PedidoResponse	pedidos.dto	37	100%
OpenApiConfig	pedidos.config	31	0%
ApiException	pedidos.exception	22	100%
UpdatePedidoRequest	pedidos.dto	9	100%
CreatePedidoRequest	pedidos.dto	9	100%
MsPedidosApplication	pedidos	8	0%
HealthController	pedidos.web	7	100%

Nota: la lógica de negocio (clases *Service*) supera el 95% de cobertura. El resto del porcentaje corresponde a configuración (CORS, OpenAPI), clase main y entidades, que no son objeto de pruebas unitarias.

5. Ejemplos de pruebas

Validación de regla de negocio (precio por plan) en ms-pedidos:

```
@Test void createOk() {
    when(repository.save(any())).thenReturn(i -> i.getArgument(0));
    PedidoResponse r = service.create(new CreatePedidoRequest(userId, 4));
    assertThat(r.total()).isEqualTo(59990);
    assertThat(r.status()).isEqualTo("PENDIENTE");
}
```

Manejo de credenciales inválidas en ms-usuarios:

```
@Test void loginWrongPassword() {
    when(repository.findByEmail(...)).thenReturn(Optional.of(usuario));
    when(passwordEncoder.matches("mala", "hash")).thenReturn(false);
    assertThatThrownBy(() -> service.login(req))
        .isInstanceOf(ApiException.class); // HTTP 401
}
```

6. Cómo ejecutar las pruebas y generar el reporte

Desde la carpeta de cada microservicio (no requiere instalar Maven, se usa el Maven Wrapper):

```
cd ms-usuarios # o ms-pedidos
.\mvnw.cmd test # Windows (./mvnw test en Linux/Mac)
```

El reporte de cobertura HTML queda en target/site/jacoco/index.html.

7. Conclusión

El sistema cuenta con 45 pruebas unitarias que cubren la lógica de negocio, la capa web y el manejo de errores de ambos microservicios. La cobertura global (83% y 79% de instrucciones) supera el umbral del 60%, garantizando la calidad y mantenibilidad del código mediante el uso de mocks (patrón de diseño aplicado a pruebas) que aíslan cada componente.